UNIVERSIDAD DE LOS ANDES

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

LABORATORIO #3: Informe técnico - Cifrado Asimétrico

ISIS 3311- Ciberseguridad

Grupo 1

Santiago Tenjo 202113965

Jose Rojas 202313911

Samuel Rodríguez Torres 202310140

Contenido

1. Implementación Cifrado Asimétrico - RSA……..3
2. . Implementación Intercambio claves - Diffie Hellman…….5
3. Pruebas y pasos de ejecución……....8
4. Comparación y análisis…….9
4.1Simétrico vs Asimétrico……..9

4.2. RSA vs Diffie-Hellman……10

4.3. Identificación de riesgos y limitaciones…..……..11
5. Recomendación……11
6. Preguntas……11
7. Referencias……....13

IMPORTANTE: El sistema funciona en ambas máquinas; únicamente es necesario cambiar los archivos que se ejecutan según el rol correspondiente (Según quien quiera enviar el archivo).

1. Implementación Cifrado Asimétrico - RSA

RSA es un algoritmo de cifrado asimétrico, lo que significa que cada usuario trabaja con un par de llaves relacionadas matemáticamente: una llave pública y una llave privada. A diferencia del cifrado simétrico, no se utiliza la misma llave para cifrar y descifrar. En RSA, una de las llaves puede usarse para cifrar y la otra para descifrar (o viceversa, según el uso), lo que permite no solo proteger información, sino también implementar mecanismos como la autenticación y la firma digital. En este sentido, si student1 y student2 quieren establecer un canal asimétrico (tipo sobre digital), cada uno debe generar su propio par de llaves en su máquina. Para ello, se implementó el proceso de la siguiente manera:

openssl genpkey -algorithm RSA -aes-256-cbc -out private.pem -pkeyopt rsa_keygen_bits:2048

Este comando desde cada uno de los usuarios permite crear una llave privada con una passphrase, esto se hace con el fin de simular un escenario real donde se tenga que custodiar la llave privada y se debe salvaguardar con la máxima seguridad posible, luego a partir de esta llave se genera la llave pública con el siguiente comando:

openssl rsa -pubout -in private.pem -out public.pem

Finalmente, se protege aún más la llave privada restringiendo los permisos que los usuarios tienen sobre esta:

chmod 600 private.pem

Con esto, se cumplió el primer paso de **generar** las llaves y **custodiar** la llave privada.

```
student@student:~$ openssl genpkey -algorithm RSA -aes-256-cbc -out private.pem -pkeyopt rsa_keygen_bits:2048
....+ … + .. +.......+.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*............+.++++++++++++++++++++
...............+......+ .. +.......+......+.........+......++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
......+.+ … +......+ … + .. +......+.......................+.++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.
++++++++++++++++++++++++++++++++++++++++++++++++++++*.. +.....+.................+..............+.+ … + .. ++++++++++++++++++++++++
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
student@student:~$ openssl rsa -pubout -in private.pem -out public.pem
Enter pass phrase for private.pem:
writing RSA key
student@student:~$ chmod 600 private.pem
student@student:~$ ls -l
total 8
-rw------- 1 student student 1874 feb 21 18:04 private.pem
-rw-rw-r-- 1 student student  451 feb 21 18:04 public.pem
```

Imagen 1: Generación y custodia de llaves para student1

Como segundo paso, necesitamos de un script en python que pueda cifrar y descifrar usando las llaves asimétricas generadas, para esto, creamos el archivo RSA.py, el cual es un archivo de python con funciones para cifrar y descifrar con la llave pública y la llave privada.

```
student@student:~/Lab3_Ciber$ python3 'Parte 1 - Asimetrico'/RSA.py
Cifrado OK. Bytes: 256
Descifrado: Hola, probando RSA con llaves generadas en OpenSSL
Firma generada. Bytes: 256
¿Firma válida? True
```

Imagen 2: Prueba de cifrado y descifrado con RSA.py

En tercer lugar, para empezar a trabajar con ambas máquinas, es necesario compartir la clave pública, de manera normal, esta clave se podría preguntar o buscar en algún directorio online. Pero para este caso, se optó por pasarlo por scp entre las máquinas con el comando:

scp student@192.101.30.10:/public.pem .

scp student@192.101.30.11:/public.pem .

Durante la ejecución del archivo [remitene.py]() y [destinatario.py]() también se compartió la llave pública de ambos usuarios con el fin de mostrar en una captura de wireshark el intercambio de claves públicas, ya que en scp los paquetes van cifrados sobre ssh.

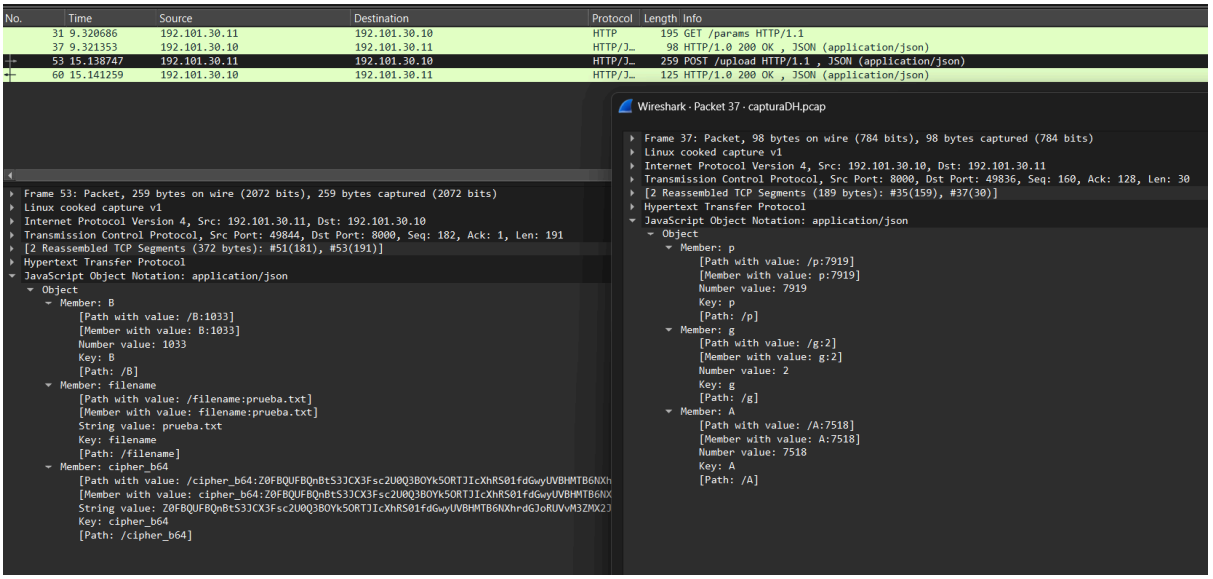


Imagen 3: Tráfico interceptado, llaves públicas compartidas.

RSA es un algoritmo de cifrado asimétrico con un costo computacional relativamente alto, por lo que no resulta eficiente para cifrar archivos grandes ni comunicaciones prolongadas. Por esta razón, se utiliza un esquema híbrido propuesto en el enunciado **(sobre digital)**. Una vez el remitente (OTHER.11) cuenta con la llave pública del destinatario (OTHER.10), genera una clave simétrica aleatoria y la cifra con RSA usando dicha llave pública. Luego, esa clave simétrica se emplea para cifrar el contenido real mediante el algoritmo simétrico AES, que es mucho más rápido y eficiente. Finalmente, el destinatario usa su llave privada para descifrar la clave simétrica y, con ella, descifra el contenido también. De esta forma, el sobre digital combina la seguridad del cifrado asimétrico con la eficiencia del cifrado simétrico.

Imagen 4: Ejecución [remitente.py]() y [destinatario.py]()

Como se puede ver en los archivos .py y en la Imagen 4, s_shared corresponde a la clave generada aleatoriamente que se usará en AES para la comunicación de los mensajes y archivos de aquí en

adelante, se imprime únicamente con fines ilustrativos, para mostrar que desde el cliente (.11) se genera y en el destino (.10) se descifra con la privada custodiada por el usuario.

Durante la ejecución de este programa, se aprovechó la implementación del laboratorio anterior y se reutilizó el archivo [AES.py](AES.py) para realizar el cifrado y descifrado de archivos o mensajes. Este módulo recibe como entrada la llave correspondiente y el contenido que se desea cifrar o descifrar (por ejemplo, archivos en formato .crypt). Una prueba de la comunicación es la captura en wireshark realizada desde una terminal aparte con tcpdump como se puede ver a continuación:

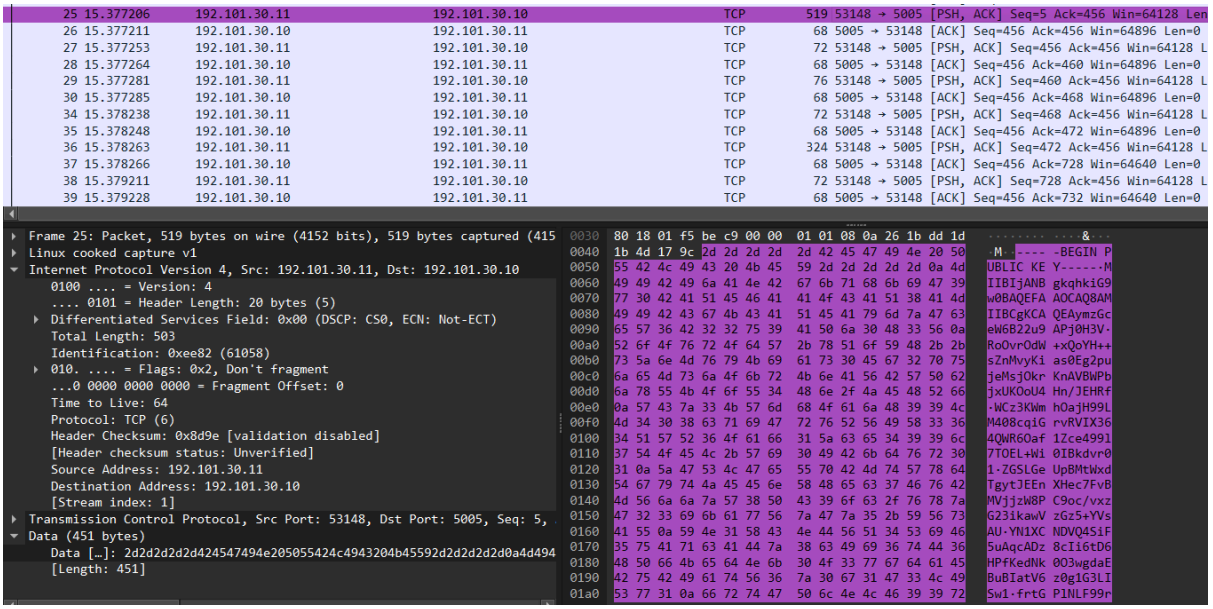


Imagen 5: Captura de tráfico RSA - Public Key.

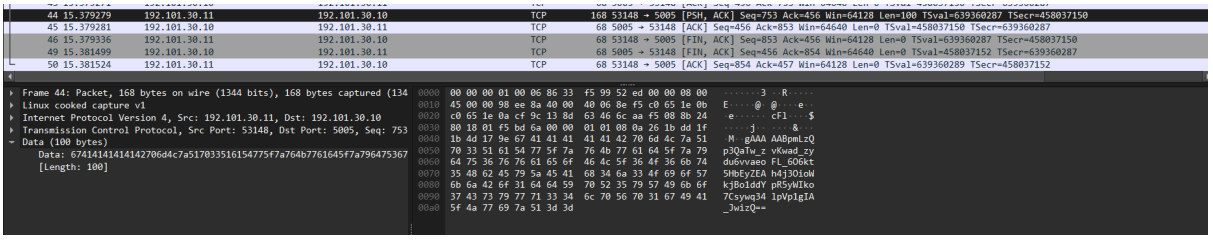


Imagen 6: Captura de tráfico RSA - Archivo cifrado.

Como se puede observar, la clave pública puede ser interceptada mediante sniffing o incluso sustituida en un ataque de man-in-the-middle si no existe un mecanismo de autenticación. En cambio, el resto de la información se transmite cifrada, por lo que solo puede ser descifrada por el servidor destinatario o por quien posea la clave privada correspondiente.

2. Implementación Intercambio claves - Diffie Hellman.

A diferencia de la implementación presentada del sobre digital y RSA, para este caso tomamos la sugerencia del profesor e implementamos un servidor HTTP con un cliente para hacer la comunicación entre ambos usuarios.

En este sentido, se buscó representar la siguiente imagen sobre la comunicación:

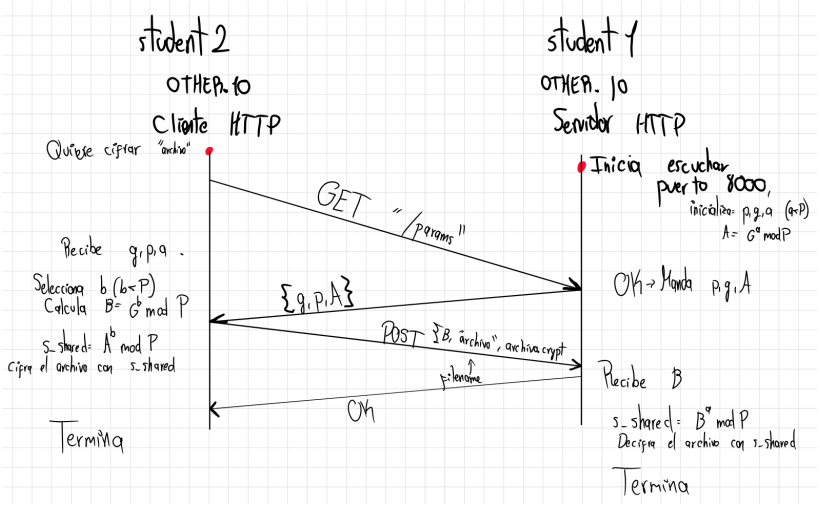


Imagen 7: Intercambio de mensajes entre student1 y student2.

Este escenario es el mismo implementado en ambos equipos en los archivos [student1.py]() y [student2.py]() dentro de la carpeta ‘Parte 2 - DH’, siguiendo las ecuaciones mostradas en la imagen 7 que en python, por ejemplo, se pueden efectuar de la siguiente manera:

$$A = pow(G, a, P)$$

La ejecución del servidor y el cliente completa se puede ver en la siguiente imagen y se puede corroborar el flujo en la comunicación de la imagen 7:

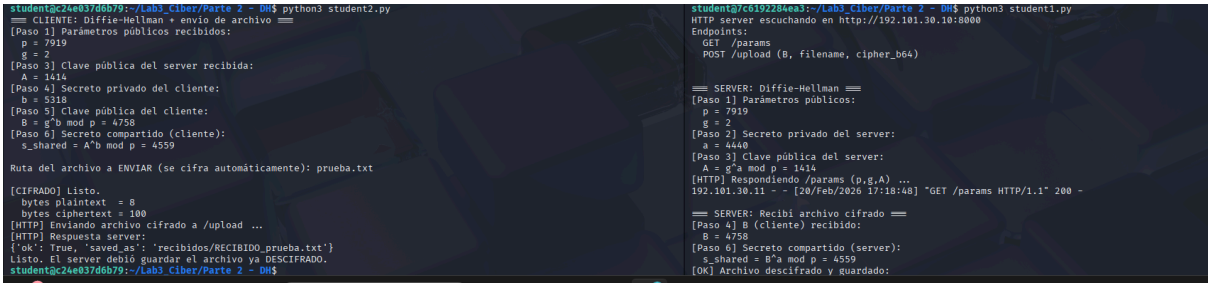
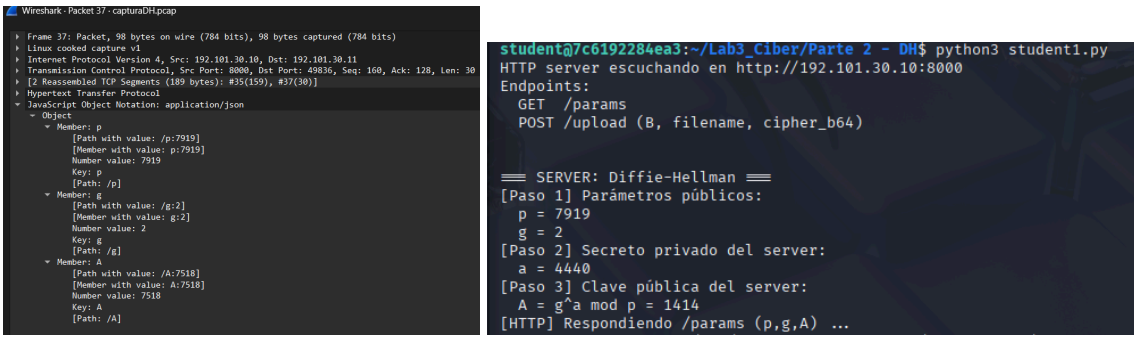


Imagen 8: Implementación de Diffie Hellman en las VM.

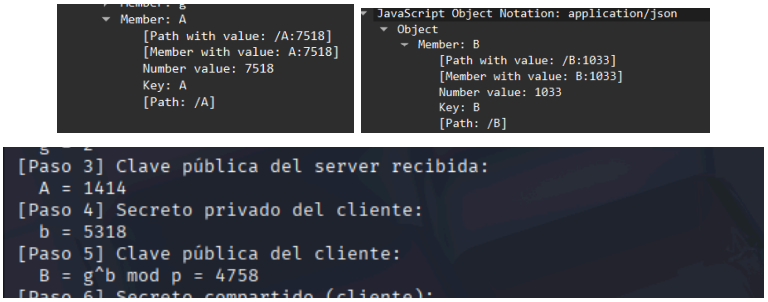
Los pasos mostrados en Imagen 4 y 5 en esencia son:

- **Generación de parámetros iniciales:** En las imágenes, se ve el paso 1 y los parámetros p y g generados por el servidor (7919, 2), dentro de cada usuario se imprime a y b (4440, 5318) aunque estos no se envían.



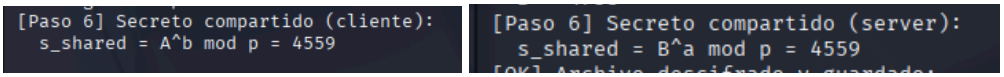
Grupo Imagen 9: Generación de parámetros iniciales.

- **Cálculo de llaves e intercambio:** Con estos valores, cada usuario calcula A y B respectivamente, se imprimen y se envían al otro usuario.



Grupo Imagen 10: Cálculo e intercambio de llaves

- **Cálculo de s_shared:** Una vez que cada cliente tiene p, g y las claves parciales A y B, ambos calculan la llave compartida s_shared a partir de los parámetros compartidos, las claves parciales y su número secreto a para student1 y b para student2.



Grupo Imagen 11: Ambos usuarios llegan a la misma llave.

- **Uso de AES con s_shared:** El cliente, en este caso student2.py, es el cliente y el que solicita la ruta del archivo a enviar; se ubica, se utiliza el algoritmo de cifrado AES del laboratorio anterior y se envía un POST a student1.py con el nombre del archivo y los bytes del archivo cifrado para que student1.py decifre el archivo y lo guarde.

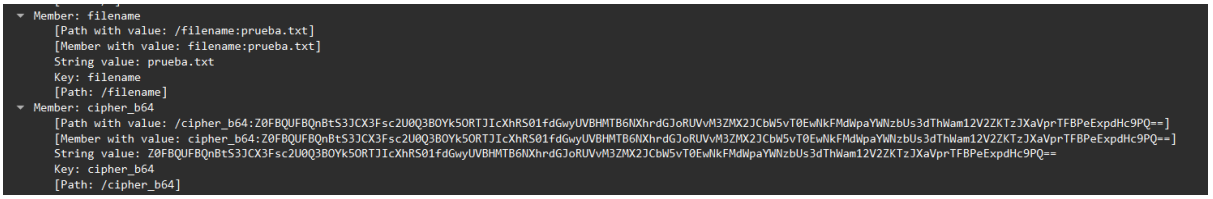


Imagen 12: El cliente envíe el archivo cifrado con AES y s_shared.

```
student@student:~/Lab3_Ciber/Parte 2 - DH$ cat RECIBIDO_prueba.txt
Prueba de seguridad con DH
student@student:~/Lab3_Ciber/Parte 2 - DH$
```

Imagen 13: Archivo descifrado en el lado del servidor.

Durante la ejecución, se pueden identificar los print() dentro de cada archivo informando sobre los parámetros usados, recibidos y la comunicación entre ambos usuarios, esto se puede corroborar con la captura en wireshark efectuada con tcpdump dentro de uno de los estudiantes en una terminal alterna.

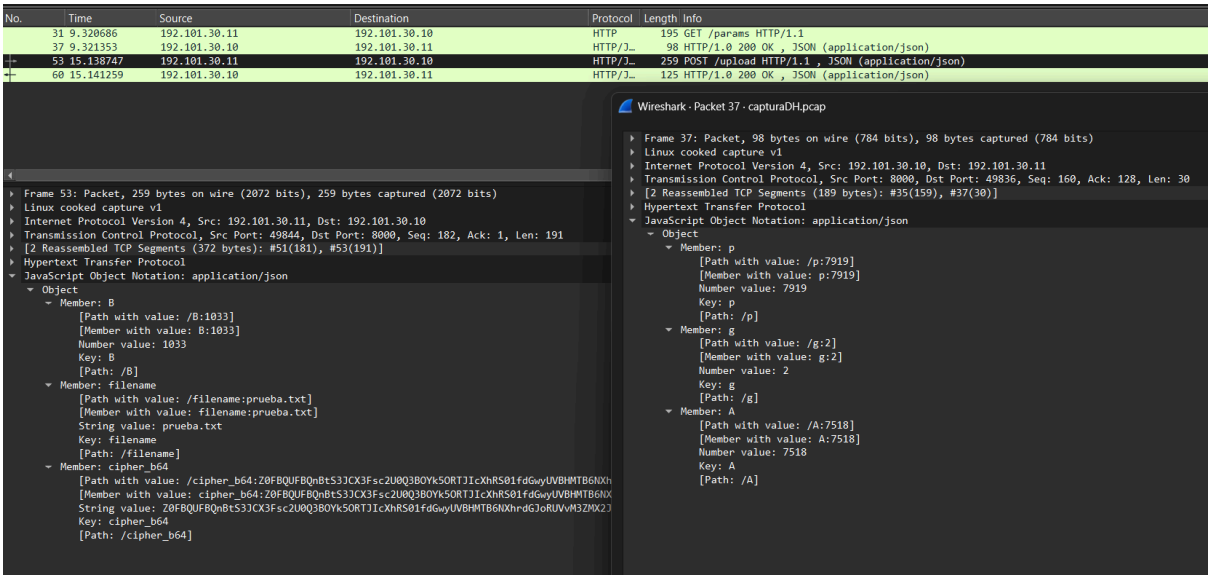


Imagen 14: Intercambio de mensajes entre student1 y student2.

3. Pruebas y pasos de ejecución:

Para evidenciar el trabajo hecho, más allá de las capturas ya presentadas, se decidió crear un repositorio en github con todo el código y las llaves generadas (aunque esto sea un gran hueco de seguridad, es con fines académicos y de evidencia) el cual se encuentra en el siguiente enlace:

https://github.com/SamuelRT123/Lab3_Ciber

Este mismo repositorio fue el que se usó dentro de las máquinas virtuales, para ejecutar el código dentro de cada una, se descargo git y con el comando git clone ya se tenía a disposición las carpetas con el código .py para ejecutar.

- **Parte 1 - Sobre Digital:** Para ejecutarlo, una de las dos máquinas debe estar usando el programa remitente.py y otro destinatario.py. Hicimos el proceso fiel a la realidad, es decir, se automatizó todo hasta el momento en el que se solitica la ruta del archivo en la máquina que ejecute remitente.py, con los print() de las partes importantes para el cambio de claves y la generación de estas.

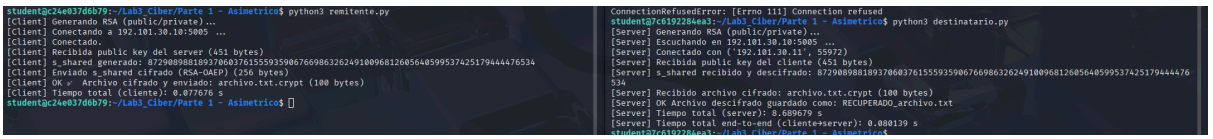


Imagen 15: Replicabilidad - Asimétrico.

- **Parte 2 - DH:** De igual manera, para ejecutar y replicar el proceso de intercambio de claves, solo basta con clonar el repositorio en ambos usuarios, ejecutar student1.py en una máquina y student2.py en la otra y todo el proceso será automático, se solicita la ruta del archivo a enviar y tanto el proceso de cifrado como de descifrado se hace automático.

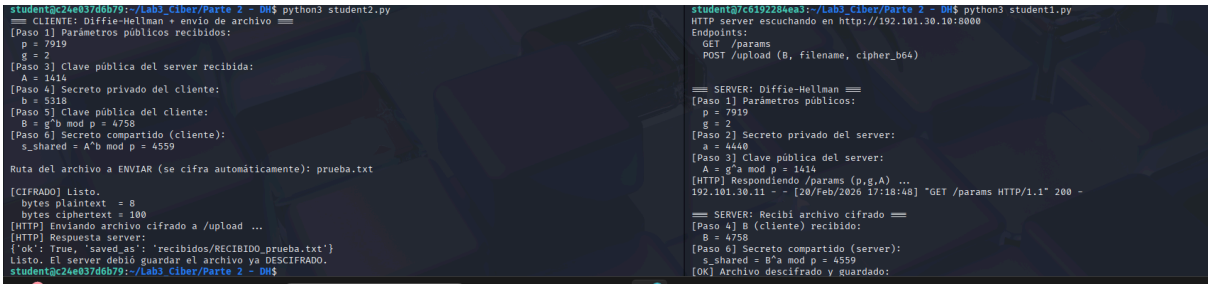


Imagen 16: Replicabilidad DH.

- **Ambiente:** Para ejecutar estas pruebas es necesario tener python, Cryptography y PyCryptodome. Y Openssl para generar las llaves públicas y privadas.

4. Comparación y análisis

4.1. Simétrico vs Asimétrico

De acuerdo con el comportamiento observado en la práctica, el cifrado **simétrico** presenta un **mejor rendimiento** para cifrar archivos y mensajes de mayor tamaño, ya que utiliza una misma clave para cifrar y descifrar y su costo computacional es mucho menor. Esto lo hace más escalable para transferencias de datos continuas o conversaciones largas. En contraste, el cifrado asimétrico (RSA) requiere operaciones matemáticas más costosas, por lo que su uso directo sobre archivos completos resulta ineficiente.

En términos de seguridad, ambos enfoques resuelven problemas distintos. El cifrado simétrico protege muy bien la confidencialidad del contenido, pero tiene como limitación principal la distribución segura de la clave: si la clave se comparte por un canal inseguro, el sistema completo queda comprometido. El cifrado asimétrico, por su parte, mejora este punto al permitir compartir una llave pública sin exponer la privada, facilitando la distribución de claves y habilitando funciones como firma digital y autenticación. Sin embargo, si no hay verificación de autenticidad de la llave pública, sigue existiendo riesgo de spoofing.

Por ello, la solución más adecuada y consistente con la teoría, el cual fue implementado durante el laboratorio fue el **sobre digital**: usar RSA para proteger una clave simétrica temporal y usar AES para cifrar el contenido real. Esta combinación aprovecha la eficiencia del cifrado simétrico y la facilidad de distribución del cifrado asimétrico.

4.2. RSA vs Diffie-Hellman

RSA y Diffie-Hellman, parte 1 y parte 2, cumplen funciones diferentes. RSA puede utilizarse para cifrado de información pequeña, para firma digital o CA, mientras que Diffie-Hellman se usa principalmente para acordar una clave compartida entre dos partes sin transmitirla directamente. En la práctica, esto significa que RSA es útil para construir un sobre digital, mientras que DH es útil para establecer una clave simétrica de sesión que luego se emplea con AES.

Desde el punto de vista de beneficios, DH evita enviar la clave simétrica por la red, ya que ambas partes la derivan localmente tras el intercambio de parámetros. RSA, en cambio, es más directo para escenarios simples: el remitente cifra una clave AES con la pública del destinatario y el destinatario la recupera con su privada. Esa simplicidad facilita la implementación y depuración.

En cuanto a riesgos y limitaciones, ambos comparten una debilidad crítica cuando no se autentican las claves/intercambios: son vulnerables a ataques de man-in-the-middle. En RSA, el atacante puede sustituir la llave pública del destinatario si no existe certificado o verificación previa. En DH, el atacante puede establecer dos claves distintas (una con cada extremo) y retransmitir mensajes, interceptando todo el tráfico. Además, en DH la seguridad depende fuertemente de usar parámetros robustos (primo grande, generadores/semillas difíciles de volver a replicar), mientras que en RSA depende del tamaño de clave, padding seguro y protección de la clave privada. En resumen: RSA es mejor para cifrado de clave y firma; DH es mejor para acuerdo de clave, pero ninguno debe usarse sin autenticación.

4.3. Identificación de riesgos y limitaciones

Un riesgo identificado en la práctica es la exposición de la llave pública al canal, que aunque no compromete por sí sola la confidencialidad, sí puede ser objeto de sniffing y, más importante aún, de sustitución maliciosa si no se valida su autenticidad. Esto abre la puerta a un ataque de man-in-the-middle, donde el atacante entrega su propia llave pública al remitente y luego descifra la clave simétrica enviada. Por tanto, la seguridad del esquema no depende solo del cifrado, sino también de cómo se distribuyen y validan las llaves públicas.

Otro riesgo importante es la custodia de la clave privada. Si la clave privada se almacena sin protección o con permisos débiles, cualquier usuario con acceso al sistema podría usarla para descifrar mensajes o suplantar identidad. En este laboratorio se planteó correctamente la protección mediante passphrase y restricciones de acceso al archivo (chmod 600), pero sigue siendo una limitación práctica si la contraseña es débil, si se deja escrita en código fuente o si se trabaja en entornos sincronizados.

Finalmente, existen limitaciones propias del diseño del laboratorio cuando se utiliza Diffie-Hellman para acordar la clave simétrica. Aunque DH suele ser una alternativa más eficiente que RSA para el establecimiento de una clave compartida, no proporciona autenticación por sí mismo; por ello, si los valores intercambiados no se validan, es vulnerable a ataques de man-in-the-middle, donde un atacante puede negociar claves distintas con cada extremo sin ser detectado. Además, su seguridad depende del uso de parámetros robustos y de una correcta generación de valores aleatorios. En consecuencia, el laboratorio demuestra las limitaciones y aplicabilidad de ambas, siendo el cifrado

asimétrico únicamente para cifrar los datos sensibles y críticos (la clave) mientras que el uso de clave simétrica para el envío de archivo. De la misma manera y como se verá en el desarrollo de las preguntas de análisis, el rendimiento de ambos limita a que equipos físicos e información debe ser utilizado.

5. Recomendación

Como recomendación final, en un contexto hospitalario se debe asumir que la comunicación entre los usuarios puede incluir datos sensibles (aunque estén en la red OTHER, no sé específica para que se quieren comunicar y fui una petición de las directivas del hospital), por lo que se debe priorizar la confidencialidad, integridad y autenticación. Por ello, se recomienda establecer un canal seguro mediante un esquema de sobre digital, en el que la infraestructura de llave pública permita proteger el intercambio inicial y reducir el riesgo de interceptación o manipulación de la comunicación.

Una mejora importante frente a generar la clave AES completamente al azar en un solo extremo es usar Diffie-Hellman (DH) para construir una clave compartida de sesión, ya que DH es adecuado y más eficiente que RSA para este propósito. En este enfoque, los elementos del intercambio (como p, g, A, B) pueden protegerse con la llave pública del destinatario durante el establecimiento del canal para reforzar la seguridad del intercambio; luego, con la clave compartida derivada, se cifra el contenido real con AES. Así, se combina la protección del esquema asimétrico con la eficiencia del cifrado simétrico, lo cual resulta más apropiado para entornos hospitalarios.

6. Preguntas

1. **¿Qué ventajas de rendimiento y escalabilidad ofrece el cifrado simétrico frente al asimétrico?**

La principal ventaja de rendimiento se basa en el costo computacional de los algoritmos, los algoritmos de AES utilizan mucho menos recursos que los de cifrado asimétrico, lo que lleva a un manejo de volúmenes de datos mucho más flexible y amplio que en el caso del cifrado asimétrico. Además de esta ventaja, podemos ser más específicos y mencionar bajos costos en la CPU, menor latencia, uso en sistemas distribuidos (IoT) y sistemas masivos. Sin embargo, esto no es una razón para utilizar siempre el cifrado simétrico, dado que el cifrado asimétrico es muchísimo más efectivo con la gestión de las claves, por lo que lo recomendable es emplear un enfoque híbrido. Esto se puede corroborar con los resultados obtenidos durante la práctica, en la que se midió el tiempo de ejecución de cada opción y se evidenció una mayor eficiencia del cifrado simétrico (Imágenes 4 y 8).

2. **¿Cómo impacta la gestión de claves (rotación, almacenamiento, distribución) en uno y otro enfoque? -**La gestión de claves es el talón de aquiles de las organizaciones, debido a que la responsabilidad de ellas recae en los humanos. Si partimos del hecho que la mayoría de los ataques a las empresas son del tipo phishing y de ingeniería social podemos asumir que el mayor riesgo de seguridad y ciberseguridad está directamente relacionado con el error humano, y si la rotación, almacenamiento y distribución depende de un humano la tendencia es que se

cometan errores y así se pierda todo el esfuerzo en generar claves y esquemas seguros.

3. ¿En qué escenarios organizacionales usaría exclusivamente simétrico, exclusivamente asimétrico o una combinación?

Usaría exclusivamente simétricos en escenarios dónde se deba cifrar un volumen de datos muy alto y quiero minimizar costos, además de garantizar pocas entidades involucradas y un entorno cerrado para disminuir la volatilidad de la gestión de claves. Por otro lado, usaría solo asimétrico para autenticar, en una firma digital o en un intercambio pequeño de información. Ahora, usaría una combinación en casi cualquier contexto, cómo mencionamos en la primera pregunta este es el enfoque estándar, es usado en HTTPS/TLS, servicios de cloud,etc. Este modelo utiliza el cifrado asimétrico para autenticar y un cifrado simétrico para proteger el tráfico masivo de datos.

4. ¿Cuál es el objetivo primario de RSA y cuál el de DH? ¿Por qué no son intercambiables?

El objetivo primario de RSA, como vimos anteriormente, es proporcionar cifrado asimétrico y firma digital, lo que permite autenticar y, además, garantizar la confidencialidad. Por otro lado, Diffie-Hellman (DH) busca establecer una clave compartida a través de un canal inseguro. Su función principal es permitir que dos partes generen una clave secreta común sin haberla compartido previamente. No son intercambiables porque cumplen funciones distintas, se basan en estructuras matemáticas diferentes y ofrecen propiedades de seguridad diferentes; por ejemplo, DH puede proporcionar forward secrecy (en esquemas efímeros), mientras que RSA por sí solo no.

5. ¿Por qué se llega a la misma clave compartida al utilizar Diffie-Hellman?

Por la definición del método y los parámetros. Primero se define un primo público p y un generador g también público. Alice y Juanchito escogen un número secreto privado, a y b respectivamente. Cada uno calcula su valor público. M y N son iguales a g**a mod p y g**b mod p respectivamente y luego intercambian M y N. Alice calcula un K = N**a mod p que es igual a (g**b)**a y Bob calcula el mismo valor K1 = M**b mod p = (g**a)**b. Por propiedades de los exponentes K es igual a K1. Obtienen el mismo valor.

Además, el uso del módulo no rompe esta igualdad, porque la aritmética modular conserva las operaciones de multiplicación y potenciación (trabajando en congruencias módulo p), es decir, reducir un valor “mod p” en pasos intermedios o al final produce resultados equivalentes dentro del mismo grupo. Por eso ambos obtienen exactamente la misma clave compartida sin haberla transmitido directamente por el canal. Lo único que se intercambia son M y N (valores públicos), mientras que a y b nunca se revelan. Matemáticamente, recuperar a, a partir de las operaciones inversas del módulo corresponde al problema del logaritmo discreto, que es computacionalmente difícil cuando p es suficientemente grande, y esa es la base de seguridad del método.

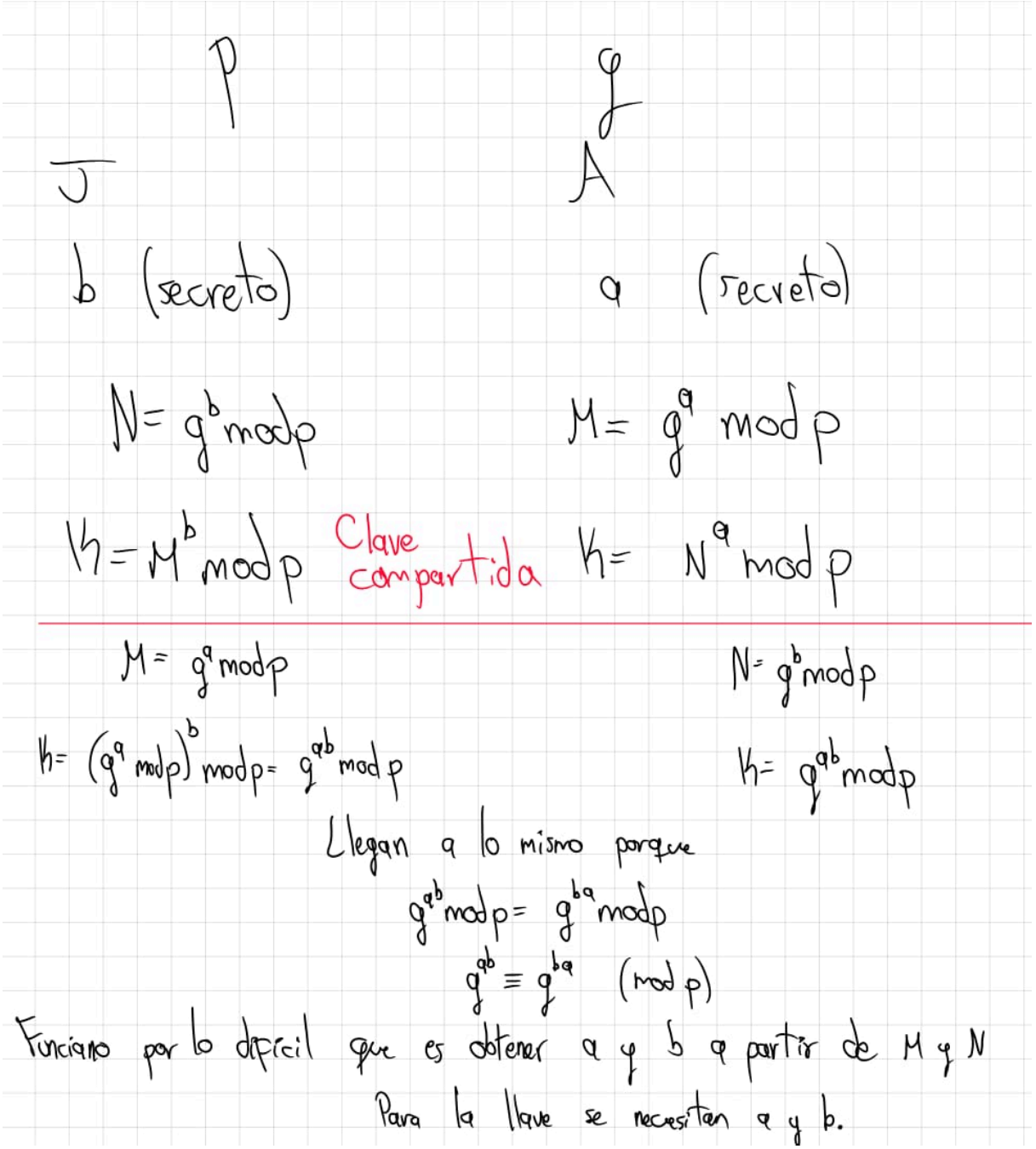


Imagen 17: Ambos usuarios llegan a la misma llave.

7. Referencias

- Presentaciones de clase
- GnuPG (GPG) The GnuPG Project. (s. f.). Documentation. Recuperado el 21 de febrero de 2026, de gnupg.org
- OpenSSL Project. (s. f.). OpenSSL documentation. Recuperado el 21 de febrero de 2026, de openssl.org
- Diffie-Hellman (YouTube) [Video sobre Diffie-Hellman] [Video]. YouTube. Recuperado el 21 de febrero de 2026, de https://www.youtube.com/watch?v=M-0qt6tdHzk

- PyCryptodome (documentación) PyCryptodome contributors. (s. f.). PyCryptodome documentation. Recuperado el 21 de febrero de 2026, de pycryptodome.readthedocs.io
- Cryptography (pyca/cryptography) (documentación) Python Cryptographic Authority. (s. f.). Cryptography documentation. Recuperado el 21 de febrero de 2026, de cryptography.io